

5.5.9

Rushil Shanmukha Srinivas
EE25BTECH11057
Electrical Engineering ,
IIT Hyderabad.

October 6, 2025

1 Problem

2 Solution

- Augmented Matrix and Inverse of a matrix

3 C Code

4 Python Code

Problem Statement

Question : Using elementary row transformations find the inverse of the matrix

$$\begin{pmatrix} 3 & 0 & -1 \\ 2 & 3 & 0 \\ 0 & 4 & 1 \end{pmatrix} \quad (2.1)$$

Augmented Matrix and Inverse of a matrix

Solution:

Given,

$$\mathbf{A} = \begin{pmatrix} 3 & 0 & -1 \\ 2 & 3 & 0 \\ 0 & 4 & 1 \end{pmatrix} \quad (3.1)$$

Forming the Augmented Matrix

$$\left(\begin{array}{ccc|ccc} 3 & 0 & -1 & 1 & 0 & 0 \\ 2 & 3 & 0 & 0 & 1 & 0 \\ 0 & 4 & 1 & 0 & 0 & 1 \end{array} \right) \quad (3.2)$$

Applying elementary row operations to find the inverse

$$\left(\begin{array}{ccc|ccc} 3 & 0 & -1 & 1 & 0 & 0 \\ 2 & 3 & 0 & 0 & 1 & 0 \\ 0 & 4 & 1 & 0 & 0 & 1 \end{array} \right) \xrightarrow{R_3 \rightarrow 3R_3 - 4R_2} \left(\begin{array}{ccc|ccc} 3 & 0 & -1 & 1 & 0 & 0 \\ 2 & 3 & 0 & 0 & 1 & 0 \\ -8 & 0 & 3 & 0 & -4 & 3 \end{array} \right) \quad (3.3)$$

$$\xrightarrow{R_3 \rightarrow 3R_3 + 8R_1} \left(\begin{array}{ccc|ccc} 3 & 0 & -1 & 1 & 0 & 0 \\ 2 & 3 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 8 & -12 & 9 \end{array} \right) \quad (3.4)$$

$$\left(\begin{array}{ccc|ccc} 3 & 0 & -1 & 1 & 0 & 0 \\ 2 & 3 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 8 & -12 & 9 \end{array} \right) \xrightarrow{R_2 \rightarrow 3R_2 - 2R_1} \left(\begin{array}{ccc|ccc} 3 & 0 & -1 & 1 & 0 & 0 \\ 0 & 9 & 2 & -2 & 3 & 0 \\ 0 & 0 & 1 & 8 & -12 & 9 \end{array} \right) \quad (3.5)$$

$$\xrightarrow{R_2 \rightarrow R_2 - 2R_3} \left(\begin{array}{ccc|ccc} 3 & 0 & -1 & 1 & 0 & 0 \\ 0 & 9 & 0 & -18 & 27 & -18 \\ 0 & 0 & 1 & 8 & -12 & 9 \end{array} \right) \quad (3.6)$$

$$\left(\begin{array}{ccc|ccc} 3 & 0 & -1 & 1 & 0 & 0 \\ 0 & 9 & 0 & -18 & 27 & -18 \\ 0 & 0 & 1 & 8 & -12 & 9 \end{array} \right) \xrightarrow{R_2 \rightarrow \frac{R_2}{9}} \left(\begin{array}{ccc|ccc} 3 & 0 & -1 & 1 & 0 & 0 \\ 0 & 1 & 0 & -2 & 3 & -2 \\ 0 & 0 & 1 & 8 & -12 & 9 \end{array} \right) \quad (3.7)$$

$$\xrightarrow{R_1 \rightarrow R_1 + R_3} \left(\begin{array}{ccc|ccc} 3 & 0 & 0 & 9 & -12 & 9 \\ 0 & 1 & 0 & -2 & 3 & -2 \\ 0 & 0 & 1 & 8 & -12 & 9 \end{array} \right) \quad (3.8)$$

$$\left(\begin{array}{ccc|ccc} 3 & 0 & 0 & 9 & -12 & 9 \\ 0 & 1 & 0 & -2 & 3 & -2 \\ 0 & 0 & 1 & 8 & -12 & 9 \end{array} \right) \xrightarrow{R_1 \rightarrow \frac{R_1}{3}} \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 3 & -4 & 3 \\ 0 & 1 & 0 & -2 & 3 & -2 \\ 0 & 0 & 1 & 8 & -12 & 9 \end{array} \right) \quad (3.9)$$

The right side part of the Augmented Matrix is $\mathbf{A}^{-1}$

$$\mathbf{A}^{-1} = \begin{pmatrix} 3 & -4 & 3 \\ -2 & 3 & -2 \\ 8 & -12 & 9 \end{pmatrix} \quad (3.10)$$

C Code

```
#include <stdio.h>
#include <math.h>
// Function to display an augmented matrix  $[A|I]$ 
void display(float mat[SIZE][2*SIZE]) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < 2*SIZE; j++) {
            printf("%10.6f", mat[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

int main() {
    // Augmented matrix  $[A|I]$  for  $A = \{\{3,0,-1\},\{2,3,0\},\{0,4,1\}\}$ 
```

```

float mat[SIZE][2*SIZE] = {
    {3.0f, 0.0f, -1.0f, 1.0f, 0.0f, 0.0f},
    {2.0f, 3.0f, 0.0f, 0.0f, 1.0f, 0.0f},
    {0.0f, 4.0f, 1.0f, 0.0f, 0.0f, 1.0f}
};
printf("Initial augmented matrix [A | I]:\n");
display(mat);
// Gauss-Jordan elimination
for (int i = 0; i < SIZE; i++) {
    // Pivot element
    float pivot = mat[i][i];
    if (fabs(pivot) < 1e-6) {
        printf("Matrix is singular and cannot be inverted.\n");
        return 1;
    }
    // Normalize pivot row
    for (int j = 0; j < 2*SIZE; j++) {
        mat[i][j] /= pivot;
    }
}

```



```

// Eliminate column entries in other rows
for (int r = 0; r < SIZE; r++) {
    if (r != i) {
        float factor = mat[r][i];
        for (int c = 0; c < 2*SIZE; c++) {
            mat[r][c] -= factor * mat[i][c];
        }
    }
    printf(" After step %d:\n", i+1);
    display(mat);
}

```

```

// Extract inverse from right half
printf(" Inverse matrix A^{-1}:\n");
for (int i = 0; i < SIZE; i++) {
    for (int j = SIZE; j < 2*SIZE; j++) {
        // Round if close to integer
        float v = mat[i][j];
    }
}

```

```
        if (fabs(v - roundf(v)) < 1e-6f)
            printf("%6.0f ", roundf(v));
        else
            printf("%8.6f ", v);
    }
    printf("\n");
}

return 0;
}
```

Python Code 1: call_c.py

```
import ctypes as ct
import numpy as np
handc1 = ct.CDLL("./func.so")
handc1.inv.argtypes = [
    ct.POINTER(ct.c_double),
    ct.POINTER(ct.c_double),
    ct.POINTER(ct.c_double)
]
A = np.array([3 , 2 , 0 ], dtype = np.float64).reshape(-1,1)
B = np.array([0 , 3 , 4 ], dtype = np.float64).reshape(-1,1)
C = np.array([-1 , 0 , 1 ], dtype = np.float64).reshape(-1,1)
handc1.inv.restype = None
handc1.inv(
    A.ctypes.data_as(ct.POINTER(ct.c_double)),
    B.ctypes.data_as(ct.POINTER(ct.c_double)),
    C.ctypes.data_as(ct.POINTER(ct.c_double)))
```

Python Code 2

```
import numpy as np
import numpy.linalg as LA
A = np.array([[3,0,-1],
              [2,3,0],
              [0,4,1]])

A_inv = LA.inv(A)

print(" Matrix-A:")
print(A)
print(" \nInverse-of-A:")
print(A_inv)
```